Aplikacja patrolowa dla DJI Mini 4 Pro

Architektura i plan działania

Aplikacja Android w Kotlinie, oparta na DJI Mobile SDK V5
Wzorce zaczerpnięte z repozytorium `dji-sdk/Mobile-SDK-Android-V5`

Wersja dokumentu	0.1 (szkic do dyskusji)
Data	3 listopada 2026
SDK	MSDK V5 `5.18.0`, WPMZ SDK `1.0.5.1`
Repozytorium	commit `91bf2d9` (3.11.2026)
Platforma	DJI Mini 4 Pro + DJI RC-N2 + telefon Android (arm64)

Spis treści

1 Cel i założenia **2**

1.1 Dlaczego aplikacja, a nie program na dronie 2

1.2 Kontekst systemu 2

1.3 Założenia sprzętowe 2

2 Co bierzemy z repozytorium **2**

3 Architektura **3**

3.1 Warstwy 3

3.2 Zasady projektowe 4

3.3 Struktura projektu 5

3.4 Zależności 5

4 Cykl pracy — maszyna stanów **5**

5 Moduły aplikacji **6**

5.1 Połączenie z dronem 7

5.2 Telemetria 7

5.3 Misja patrolu 8

5.4 Wizja i markery 9

5.5 Lądowanie 9

5.6 Sterowanie lotem (Virtual Stick) 10

5.7 Łącze ze stacją 11

5.8 Nadzór i STOP 11

6 Ekran aplikacji **12**

7 Testy wykonalności R1–R7 **13**

8 Plan działania **13**

9 Ryzyka **16**

A Ściąga API MSDK użyte w projekcie **17**

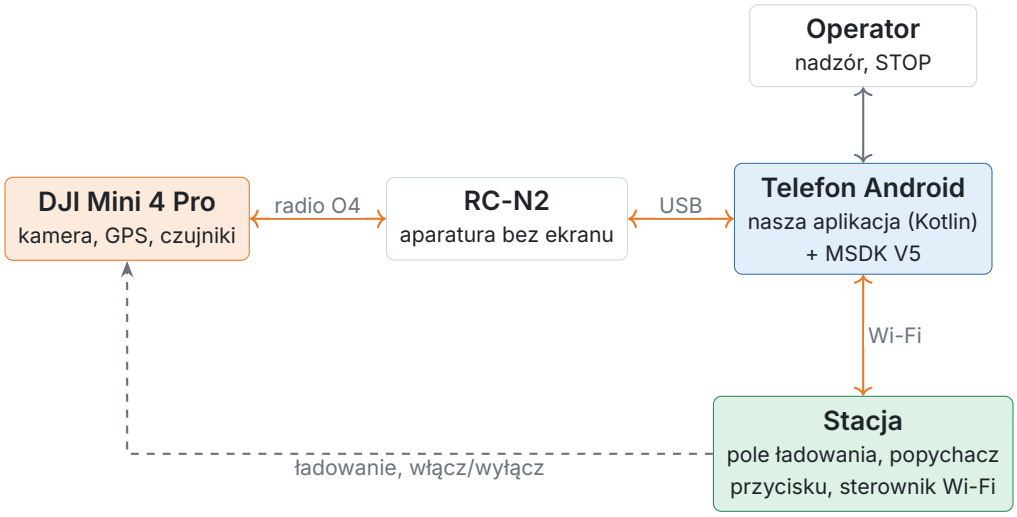
1 Cel i założenia

Budujemy aplikację, która samodzielnie prowadzi pełny cykl pracy drona: start ze stacji, patrol po zaplanowanej trasie, powrót, precyzyjne lądowanie na markerze, wyłączenie i ładowanie. Operator nadzoruje cykl i może go w każdej chwili zatrzymać.

1.1 Dlaczego aplikacja, a nie program na dronie

1. **Dron jest zamknięty.** Mini 4 Pro nie pozwala uruchamiać własnego kodu na pokładzie (nie ma Onboard/PSDK).
2. **Sterowanie z telefonu przy pilocie RC-N2.** Jedyną otwartą drogą jest Mobile SDK V5 na telefonie z Androidem połączonym kablem z aparaturą.
3. **Aplikacja prowadzi cały cykl.** Cała logika (misja, wizja, lądowanie, decyzje o powrocie, komunikacja ze stacją) działa w aplikacji.

1.2 Kontekst systemu



1.3 Założenia sprzętowe

- **Aparatura DJI RC-N2.** MSDK z Mini 4 Pro działa wyłącznie z RC-N2. Aparatura RC 2 (z ekranem) nie pozwala instalować aplikacji firm trzecich.
- **Telefon z Androidem, architektura arm64-v8a, Android 7.0+** (`minSdk24`, jak w repo). Nie ma wersji na iOS.
- **Telefon łączy się z aparaturą przez USB**, więc jego Wi-Fi jest wolne do komunikacji ze stacją.
- **Ładowanie przy wyłączonym dronie** (ustalone w teście R1) oraz **włączanie/wyłączanie popychaczem przycisku** na stacji (ustalone w teście R2).
- **Pierwsza rejestracja aplikacji w MSDK wymaga internetu** (weryfikacja klucza API).

2 Co bierzemy z repozytorium

Repozytorium `Mobile-SDK-Android-V5` nie zawiera samego SDK (to zamknięte biblioteki AAR z Maven Central), lecz aplikację przykładową z jednym ekranem na każdą funkcję. Nie przerabiamy

jej. Zakładamy czysty projekt i przenosimy sprawdzone fragmenty. Poniższa tabela pokazuje, który plik jest wzorcem dla którego modułu naszej aplikacji.

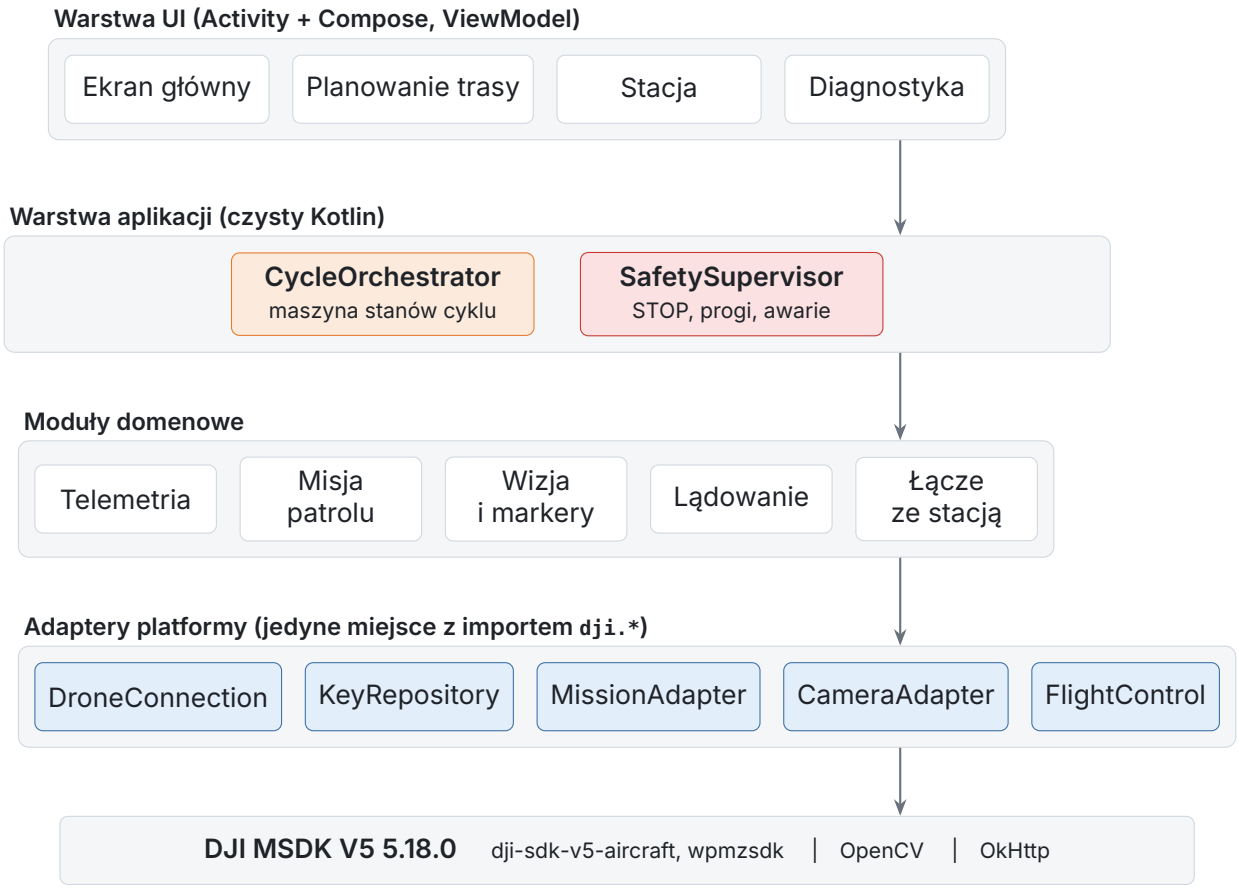
Moduł aplikacji	Wzorzec w repozytorium (`android-sdk-v5-sample/.../aircraft/`)	Główne API MSDK
Połączenie z dronem	`DJIAircraftApplication.kt`, `models/MSDKManagerVM.kt`, `UsbAttachActivity.java`	`SDKManager`
Telemetria	`models/WayPointV3VM.kt` (`listenFlightControlState`), `pages/KeyValueFragment.kt`	`KeyManager`, `FlightControllerKey`, `BatteryKey`
Misja patrolu	`models/WayPointV3VM.kt`, `pages/WayPointV3Fragment.kt`, `utils/KMZTestUtil.java`	`WaypointMissionManager`, `WPMZManager`
Wizja i markery	`models/CameraStreamDetailVM.kt` (`addFrameListener`)	`CameraStreamManager`
Sterowanie lotem	`models/VirtualStickVM.kt`, `pages/VirtualStickFragment.kt`	`VirtualStickManager`
Lądowanie	`models/BasicAircraftControlVM.kt` + własny regulator	`KeyStartAutoLanding`, `KeyConfirmLanding`
Strefy lotów	`models/FlySafeVM.kt`, `pages/FlySafeFragment.kt`	`FlyZoneManager`
Symulator	`models/SimulatorVM.kt`	`SimulatorManager`
Interfejs (wideo, mapa)	moduł `android-sdk-v5-uxsdk` (widżety FPV i mapy)	—
Łącze ze stacją	*brak w repo — własna implementacja*	OkHttp WebSocket
Nadzór i STOP	*brak w repo — własna implementacja*	`VirtualStickStateListener`, `pauseMission`

Konfiguracja projektu przeniesiona z repo Z `android-sdk-v5-as/gradle.properties` i `android-sdk-v5-sample/build.gradle` kopiujemy: `abiFilters 'arm64-v8a'`, blok `packagingOptions` z `pickFirst 'lib/arm64-v8a/libc++_shared.so'` i listą `doNotStrip` bibliotek DJI, wstrzykiwanie klucza przez `manifestPlaceholders["API_KEY"]`, wpis `com.dji.sdk.API_KEY` w manifeście, filtr `USB_ACCESSORY_ATTACHED` oraz wywołanie `com.cySdkyc.clx.Helper.install(this)` w `attachBaseContext` klasy `Application`. Usuwamy martwe repozytoria `jcenter()` i `dl.bintray.com`. Klucza testowego `msdkkeystore.jks` (hasło `123456`) nie używamy do wydań.

3 Architektura

3.1 Warstwy

Aplikacja ma cztery warstwy. Kluczowa zasada: **tylko warstwa adapterów zna MSDK**. Logika cyklu jest w czystym Kotlinie, więc testujemy ją jednostkowo bez drona, a w razie zmian w SDK poprawiamy jedno miejsce.



UI Jedno Activity, ekrany w Jetpack Compose. Podgląd wideo i mapa jako widoki Android osadzone przez `AndroidView` (podobnie jak widżety z `uxsdk`). ViewModel tylko prezentuje stan i przekazuje polecenia operatora.

Aplikacja `CycleOrchestrator` decyduje, co dron robi teraz (sekcja 4). `SafetySupervisor` ma wyższy priorytet i może przerwać każdy etap.

Moduły domenowe Każdy realizuje jedno zadanie z diagramu D2 i komunikuje się przez `Flow`.

Adaptery Opakowują singletony MSDK (`KeyManager`, `VirtualStickManager`, `WaypointMissionManager` ...) w interfejsy Kotlina z korutynami. Dla testów każdy ma wersję `Fake`.

3.2 Zasady projektowe

- **Z1. Jedno źródło prawdy o dronie.** `DroneState` to jeden `StateFlow` składany z kluczy telemetrii. UI i logika czytają tylko z niego.
- **Z2. Jeden właściciel sterowania.** W danej chwili dronem steruje dokładnie jeden mechanizm: misja WPML *albo* Virtual Stick *albo* automatyczne lądowanie DJI. `FlightControl` pilnuje przekazania (np. przed włączeniem Virtual Stick misja musi być zatrzymana).
- **Z3. Korutyny zamiast LiveData i RxJava.** Repozytorium miesza LiveData, RxJava 2/3 i callbacki. My opakowujemy callbacki w `callbackFlow` / `suspendCancellableCoroutine` i mamy jeden model współbieżności.
- **Z4. Bezpieczny stan domyślny.** Każdy błąd, utrata danych czy wyjątek kończy się zawisem lub procedurą awaryjną drona, nigdy wyłączeniem silników w powietrzu.
- **Z5. Wszystko jest logowane.** Telemetria, przejścia stanów, polecenia sterujące i wyniki detekcji markerów trafiają do lokalnego logu z czasem. Bez tego testy R3–R6 są nieweryfikowalne.

3.3 Struktura projektu

```
patrol-app/
├── app/                    UI (Compose), DI, PatrolApp : Application, manifest
├── core/                   czysty Kotlin, bez Androida i bez DJI
│   ├── model/              DroneState, CycleState, PatrolRoute, MarkerObservation
│   ├── orchestrator/       CycleOrchestrator, SafetySupervisor, BatteryPolicy
│   └── landing/            LandingController (regulator), LandingPlanner
├── msdk-adapter/           JEDYNY moduł zależny od com.dji:dji-sdk-v5-*
│   ├── connection/         SdkInitializer, DroneConnection
│   ├── telemetry/          KeyRepository (KeyManager → Flow)
│   ├── mission/            KmzBuilder (WPMZManager), MissionAdapter
│   ├── flight/             FlightControl (VirtualStickManager, takeoff/land)
│   ├── camera/             CameraAdapter (CameraStreamManager, gimbal)
│   └── flysafe/            FlyZoneChecker
├── vision/                 ArucoDetector (OpenCV), estymacja pozy
└── station/                StationClient (WebSocket), protokół JSON
```

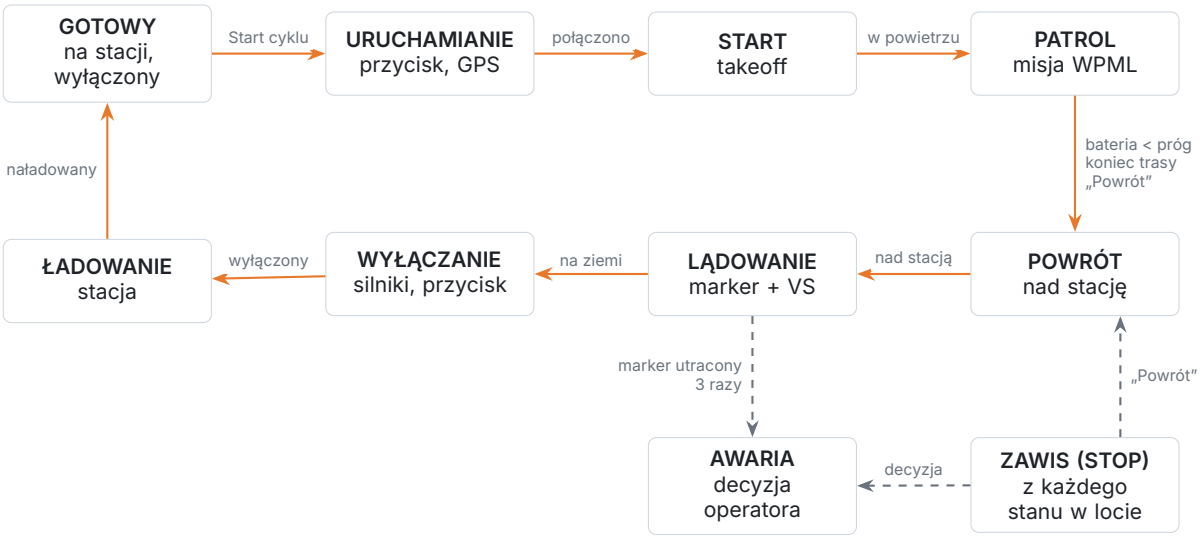
Zależności: `app` → wszystkie; `core` → nic; `msdk-adapter`, `vision`, `station` → `core`. Dzięki temu `core` można testować na JVM w sekundach.

3.4 Zależności

Biblioteka	Uwagi
`com.dji:dji-sdk-v5-aircraft:5.18.0`	`implementation`; główny pakiet MSDK
`com.dji:dji-sdk-v5-aircraft-provided:5.18.0`	`compileOnly`; interfejsy
`com.dji:dji-sdk-v5-networkImp:5.18.0`	`runtimeOnly`; bez tego nie działa rejestracja ani sieć MSDK
`com.dji:wpmzsdk:1.0.5.1`	generowanie i parsowanie plików KMZ/WPML (jak w repo)
OpenCV for Android (4.9+)	detekcja ArUco (`objdetect`); ma własne `libc++_shared.so`, stąd potrzebne `pickFirst`
Kotlin Coroutines, Jetpack Compose, Lifecycle	współbieżność i UI
MapLibre Android	mapa trasy (repo używa MapLibre lub Google Maps)
OkHttp	klient WebSocket do stacji

4 Cykl pracy — maszyna stanów

Etapy cyklu z ekranu aplikacji (Start, Patrol, Powrót, Lądowanie, Ładowanie) są stanami jednej maszyny w `CycleOrchestrator`. Każde przejście ma warunek wejścia i jest logowane.



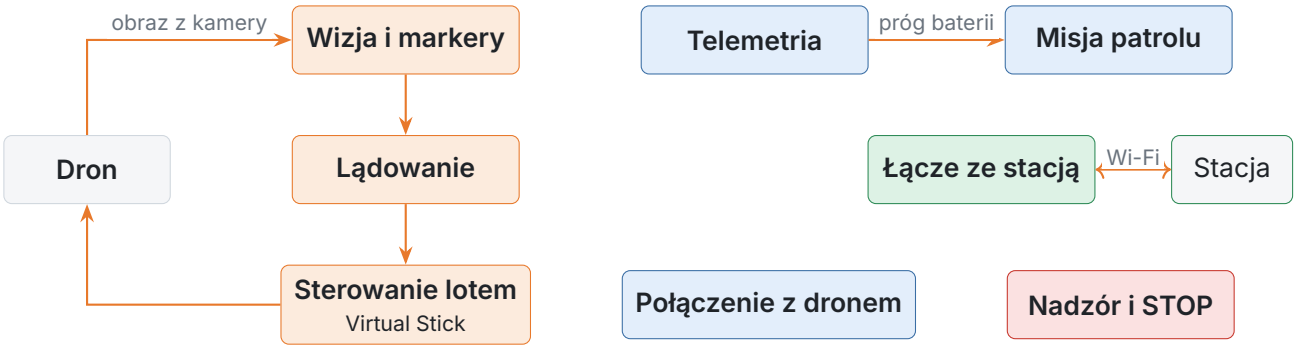
- **Punkt przerwania.** Przy przejściu PATROL → POWRÓT zapisujemy indeks ostatniego osiągniętego punktu (`WaylineExecutingInfo.currentWaypointIndex`). Następny cykl wznawia patrol od tego punktu (na ekranie: „Punkt przerwania").
- **STOP** przenosi do ZAWISU z każdego stanu w powietrzu. Z ZAWISU operator wybiera „Wznów" (powrót do poprzedniego stanu) albo „Powrót do stacji".
- **Stany naziemne** (GOTOWY, URUCHAMIANIE, ŁADOWANIE) sterują stacją, a nie dronem.

```
sealed interface CycleState {
    data object Ready : CycleState
    data object Booting : CycleState
    data object TakingOff : CycleState
    data class Patrolling(val waypoint: Int, val total: Int) : CycleState
    data class Returning(val reason: ReturnReason, val breakpoint: Int?) : CycleState
    data class Landing(val attempt: Int) : CycleState
    data object ShuttingDown : CycleState
    data object Charging : CycleState
    data class Holding(val resumeTo: CycleState) : CycleState   // STOP
    data class Failure(val cause: String) : CycleState
}

enum class ReturnReason { LOW_BATTERY, ROUTE_DONE, OPERATOR, SAFETY }
```

5 Moduły aplikacji

Moduły odpowiadają diagramowi D2. Strzałki pokazują główny przepływ danych: obraz z kamery zamienia się w polecenia lądowania, a telemetria baterii decyduje o przerwaniu patrolu.



5.1 Połączenie z dronem

Odpowiedzialność: inicjalizacja MSDK, rejestracja aplikacji, wykrycie połączenia i typu produktu, ponowne połączenie po włączeniu drona przez stację.

Przebieg jest taki sam jak w `MSDKManagerVM.kt`: `SDKManager.init()` → zdarzenie `INITIALIZE_COMPLETE` → `registerApp()` → `onProductConnect()`. Dodatkowo sprawdzamy `ProductKey.KeyProductType==DJI_MINI_4_PRO` i odrzucamy inne modele.

```
class SdkInitializer(private val ctx: Context) {
    private val _state = MutableStateFlow<SdkState>(SdkState.Initializing)
    val state: StateFlow<SdkState> = _state

    fun start() = SDKManager.getInstance().init(ctx, object : SDKManagerCallback {
        override fun onInitProcess(event: DJISDKInitEvent, total: Int) {
            if (event == DJISDKInitEvent.INITIALIZE_COMPLETE)
                SDKManager.getInstance().registerApp()   // jak w repo
        }
        override fun onRegisterSuccess() { _state.value = SdkState.Registered }
        override fun onRegisterFailure(e: IDJIError) { _state.value = SdkState.Error(e) }
        override fun onProductConnect(id: Int) { _state.value = SdkState.Connected(id) }
        override fun onProductDisconnect(id: Int) { _state.value = SdkState.Disconnected }
        override fun onProductChanged(id: Int) {}
        override fun onDatabaseDownloadProgress(cur: Long, total: Long) {}
    })
}
```

> **Uwaga** Po wyłączeniu i ponownym włączeniu drona przez stację (R2) aparatura musi odnowić łącze. Stan URUCHAMIANIE czeka na `onProductConnect`, `KeyConnection==true`, wystarczający `KeyGPSSignalLevel` i zapisany punkt startu (`KeyHomeLocation`). Dopiero wtedy pozwala na start.

5.2 Telemetria

Odpowiedzialność: zebranie wszystkich potrzebnych kluczy w jeden `DroneState` i wyznaczenie progu powrotu.

Każdy klucz zamieniamy w `Flow`. Repozytorium używa wywołania `KeyManager.getInstance().listen(key,holder){old,new->}` (np. `uxsdk/.../MapWidgetModel.java`). Opakowujemy je raz:

```
fun <T> DJIKey<T>.asFlow(): Flow<T?> = callbackFlow {
    val holder = Any()
    KeyManager.getInstance().listen(this@asFlow, holder) { _, new -> trySend(new) }
```

```
    awaitClose { KeyManager.getInstance().cancelListen(holder) }
}

val battery = KeyTools.createKey(BatteryKey.KeyChargeRemainingInPercent).asFlow()
```

Klucz MSDK	Użycie w aplikacji
`BatteryKey.KeyChargeRemainingInPercent`	bateria na ekranie, próg powrotu
`FlightControllerKey.KeyAircraftLocation`, `KeyAltitude`	pozycja na mapie, kontrola trasy
`KeyCompassHeading`, `KeyAircraftVelocity`	kurs i prędkość; dane do regulatora lądowania
`KeyUltrasonicHeight`	wysokość nad podłożem w końcowej fazie lądowania (*do weryfikacji na Mini 4 Pro*)
`KeyGPSSignalLevel`, `KeyGPSSatelliteCount`	warunek startu, wskaźnik GPS
`KeyIsFlying`, `KeyAreMotorsOn`, `KeyFlightMode`	rozpoznanie etapu i przejścia stanów
`KeyHomeLocation`	punkt startu = stacja
`KeyIsLowBatteryWarning`, `KeyIsSeriousLowBatteryWarning`	awaryjne progi drona
`KeyIsFailSafe`, `KeyFailsafeAction`	zachowanie przy utracie łącza

Próg powrotu (na ekranie: „Bateria i próg powrotu"). `BatteryPolicy` wylicza go jako stałą rezerwę na lądowanie z markerem plus energię na lot do stacji, proporcjonalną do odległości od niej. Gdy bateria spadnie poniżej progu, telemetria emituje zdarzenie `ReturnRequired` do orkiestratora. Stałe kalibrujemy na podstawie logów z testów.

5.3 Misja patrolu

Odpowiedzialność: zamiana trasy z mapy na plik KMZ, wysłanie go do drona, uruchomienie, śledzenie postępu, wznowienie od punktu przerwania.

Używamy natywnych misji waypoint (WPML), dokładnie jak `WayPointV3VM.kt`. Dron wykonuje trasę sam, więc chwilowy zanik łączności z telefonem nie przerywa lotu, a działa wbudowane omijanie przeszkód.

1. **Planowanie** (ekran „Planowanie trasy"): operator klika punkty na mapie i ustawia wysokość, prędkość, kurs i akcje (np. zdjęcie).
2. **Budowa KMZ**: `WPMZManager.getInstance().generateKMZFile(path,mission,config,template)`. Obiekty `WaylineMission`, `WaylineMissionConfig` i `WaylineWaypoint` budujemy wg `KMZTestUtil.java`. Ostatni punkt to punkt nad stacją na wysokości około 8–10 m.
3. **Walidacja**: kontrola stref lotów (R7) i limitów wysokości (`KeyHeightLimit`) przed wysłaniem.
4. **Wysłanie**: `WaypointMissionManager.pushKMZFileToAircraft(path,callback)` z postępem.
5. **Start**: `startMission(missionId,waylineIds)`. Postęp czytamy z `addWaylineExecutingInfoListener` (`currentWaypointIndex` → „Patrol · punkt 3 z 6") i `addWaypointMissionExecuteStateListener`.
6. **Przerwanie**: `pauseMission()` lub `stopMission()` i zapis punktu przerwania.

Do sprawdzenia na Mini 4 Pro Funkcje wznawiania od punktu przerwania (`BreakPointInfo`, `queryBreakPointInfoFromAircraft`) są w dokumentacji opisane głównie dla dronów Enterprise. Jeśli na Mini 4 Pro nie zadziałają, stosujemy rozwiązanie zastępcze: generujemy nowy plik KMZ z pozostałymi punktami trasy, licząc od zapisanego indeksu.

5.4 Wizja i markery

Odpowiedzialność: pobieranie klatek z kamery, wykrywanie markera ArUco na lądowisku i wyznaczanie położenia drona względem markera.

```
MediaDataCenter.getInstance().cameraStreamManager.addFrameListener(
    ComponentIndexType.LEFT_OR_MAIN,
    ICameraStreamManager.FrameFormat.NV21,
    object : ICameraStreamManager.CameraFrameListener {
        override fun onFrame(data: ByteArray, offset: Int, length: Int,
                             width: Int, height: Int,
                             format: ICameraStreamManager.FrameFormat) {
            latestFrame.offer(Frame(data.copyOfRange(offset, offset + length),
                                    width, height, SystemClock.elapsedRealtime()))
        }
    })
```

- **Kamera w dół:** przed lądowaniem gimbal na -90° (`GimbalKey.KeyRotateByAngle`); stan sprawdzamy w `KeyGimbalAttitude`.
- **Tylko najnowsza klatka:** kolejka o pojemności 1 i przetwarzanie na osobnym wątku (`Dispatchers.Default`). Stara klatka jest bezużyteczna dla regulatora.
- **Detekcja:** OpenCV `ArucoDetector`, potem `solvePnP` z kalibracją kamery → `MarkerObservation(dx,dy,dz,yaw,timestamp)`.
- **Markery zagnieżdżone:** duży marker widoczny z 8–10 m i mały w środku do ostatnich kilkudziesięciu centymetrów, bo z bliska duży marker wychodzi poza kadr.
- **Nakładka na podgląd:** ramka i celownik na obrazie („Obraz z kamery i marker" na ekranie).

Ten moduł nie ma zależności od DJI poza `CameraAdapter`, więc rozwijamy go na nagranych filmach z drona, zanim zaczniemy latać.

5.5 Lądowanie

Odpowiedzialność: sprowadzenie drona z punktu nad stacją dokładnie na pole ładowania.

Powrót do stacji wbudowaną funkcją RTH ma dokładność rzędu metrów, co nie wystarcza do trafienia w styki ładowania. Dlatego lądowanie jest wieloetapowe:

L1. Dolot: ostatni punkt misji około 8–10 m nad stacją; zatrzymanie misji.

L2. Szukanie markera: gimbal w dół, włączenie Virtual Stick, zawis. Gdy markera brak przez N sekund, wznosimy się o kilka metrów i szukamy ponownie.

L3. Centrowanie i opadanie: regulator PD w osi X/Y (prędkości w układzie `BODY`) z korekcją kursu. Opadamy tylko wtedy, gdy błąd poziomy jest poniżej tolerancji; im niżej, tym tolerancja mniejsza i prędkości mniejsze.

L4. **Przyziemienie:** na małej wysokości (np. 0,5 m) wywołujemy `KeyStartAutoLanding`. Jeśli dron zgłasza `KeyIsLandingConfirmationNeeded` (ochrona lądowania), potwierdzamy `KeyConfirmLanding` — to test R5.

L5. **Weryfikacja:** `KeyAreMotorsOn==false` i potwierdzenie ze stacji, że dron leży na polu ładowania.

Przerwanie: utrata markera przez dłużej niż ustalony czas → wznieś się i powtórz od L2; po trzech nieudanych próbach → stan AWARIA i decyzja operatora.

> **Brak omijania przeszkód w Virtual Stick** Dokumentacja `IVirtualStickManager` wymienia jako obsługujące omijanie przeszkód w trybie Virtual Stick tylko M300/M350 RTK, serię M30 oraz Mavic 3E/3M. **Mini 4 Pro nie ma na tej liście.** Virtual Stick używamy więc wyłącznie w fazie lądowania nad wolnym od przeszkód lądowiskiem, z ograniczonymi prędkościami. Patrol realizujemy natywną misją WPML.

5.6 Sterowanie lotem (Virtual Stick)

Odpowiedzialność: jedyny moduł, który wysyła polecenia ruchu: start, lądowanie, Virtual Stick, zawis.

Wzorzec to `VirtualStickVM.kt`: `enableVirtualStick()`, `setVirtualStickAdvancedModeEnabled(true)` i `sendVirtualStickAdvancedParam(param)`. Repozytorium wysyła parametry tylko po kliknięciu przycisku; my potrzebujemy pętli. Dokumentacja zaleca wysyłanie z częstotliwością **5–25 Hz**.

```
class FlightControl(private val vs: VirtualStickManager) {
    @Volatile private var target = Setpoint.HOVER
    @Volatile private var targetTime = 0L

    fun setTarget(s: Setpoint) { target = s; targetTime = SystemClock.elapsedRealtime() }

    suspend fun runLoop() = coroutineScope {
        while (isActive) {
            // Watchdog: brak świeżego polecenia przez 200 ms → zawis
            val s = if (SystemClock.elapsedRealtime() - targetTime > 200) Setpoint.HOVER
                    else target
            vs.sendVirtualStickAdvancedParam(s.toParam())
            delay(50)                                   // 20 Hz
        }
    }
}

fun Setpoint.toParam() = VirtualStickFlightControlParam().apply {
    rollPitchCoordinateSystem = FlightCoordinateSystem.BODY
    rollPitchControlMode = RollPitchControlMode.VELOCITY   // nie ANGLE jak w repo
    verticalControlMode  = VerticalControlMode.VELOCITY
    yawControlMode       = YawControlMode.ANGULAR_VELOCITY
    pitch = vx; roll = vy; verticalThrottle = vz; yaw = yawRate
}
```

- **Prędkości, nie kąty.** Repozytorium domyślnie ustawia `RollPitchControlMode.ANGLE`. Do lądowania potrzebujemy trybu `VELOCITY`: prościej się go stroi, a dokumentacja wymaga go do omijania przeszkód na modelach, które je wspierają. Mapowanie osi `pitch`/`roll` na kierunki X/Y w układzie `BODY` potwierdzamy w symulatorze.

- **Utrata kontroli.** Nasłuchujemy `VirtualStickStateListener.onChangeReasonUpdate`. Każda zmiana właściciela sterowania (np. przejęcie przez aparaturę, zbliżenie do strefy zakazanej) przerywa etap i przechodzi do ZAWISU lub AWARII.
- **Start i lądowanie** przez `KeyStartTakeoff` i `KeyStartAutoLanding` (jak `BasicAircraftControlVM.kt`).

5.7 Łącze ze stacją

Odpowiedzialność: wymiana stanu i poleceń ze stacją przez Wi-Fi. Tej części repo nie zawiera.

Telefon jest połączony z aparaturą przez USB, więc Wi-Fi pozostaje wolne. Proponujemy sterownik stacji (np. ESP32) jako punkt dostępowy lub w tej samej sieci lokalnej, protokół JSON przez WebSocket i heartbeat co 1 s.

Kierunek	Wiadomość	Znaczenie
stacja → aplikacja	`status`	`charging`, `dronePresent`, `padClear`, `ready` („Stacja gotowa" na ekranie)
aplikacja → stacja	`powerButton`	sekwencja popychacza (włącz/wyłącz; dokładną sekwencję naciśnięć ustala R2)
aplikacja → stacja	`charge(on/off)`	sterowanie ładowaniem (R1: tylko przy wyłączonym dronie)
obie strony	`ack(id)`, `heartbeat`	potwierdzenie polecenia; brak heartbeatu przez 3 s = stacja niedostępna

Każde polecenie ma identyfikator i czeka na `ack`. Polecenia są idempotentne, więc ponowienie po utracie odpowiedzi jest bezpieczne. Utrata łącza ze stacją *w locie* nie przerywa patrolu, ale blokuje lądowanie do czasu odzyskania statusu `padClear`.

5.8 Nadzór i STOP

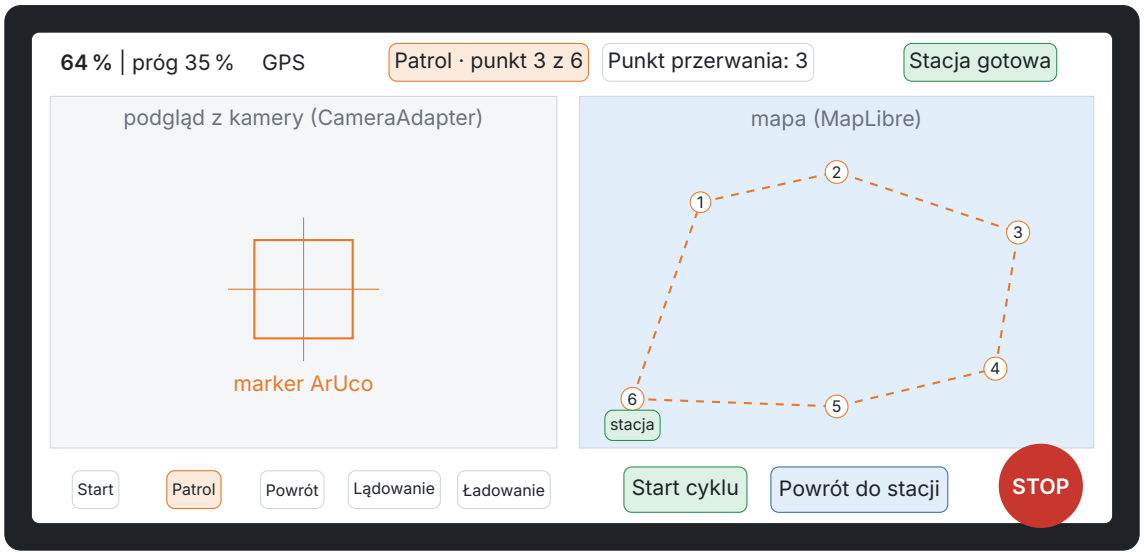
Odpowiedzialność: najwyższy priorytet w aplikacji. Może przerwać każdy etap.

Zdarzenie	Reakcja
STOP (przycisk na ekranie)	natychmiastowy zawis: `pauseMission()` w trakcie misji, zerowe prędkości w Virtual Stick, przerwanie lądowania. Nigdy nie wyłącza silników w powietrzu.
Bateria poniżej progu powrotu	PATROL → POWRÓT
Bateria krytyczna (`KeyIsSeriousLowBatteryWarning`)	lądowanie w bieżącym miejscu, alarm
Utrata łącza dron–aparatura	zachowanie drona wg `KeyFailsafeAction`; ustawiamy powrót do punktu startu (stacji)
Utrata łącza ze stacją	blokada lądowania, zawis nad stacją, alarm
Zmiana właściciela Virtual Stick	ZAWIS lub AWARIA
Zbliżenie do strefy lotów	blokada startu misji, alarm

Przejęcie ręczne Operator z aparaturą RC-N2 musi być zawsze w gotowości do przejęcia sterowania. Dokładne zachowanie aparatury przy aktywnym Virtual Stick i misji (które przyciski i drążki przejmują kontrolę) weryfikujemy w teście R3 i spisujemy w procedurze operacyjnej.

6 Ekran aplikacji

Ekran główny realizuje projekt D1. Pracujemy w orientacji poziomej, z telefonem w uchwycie RC-N2.



Element	Źródło danych / akcja
Bateria i próg powrotu	`DroneState.battery`, `BatteryPolicy.threshold`
Etap i punkt patrolu	`CycleState` + `currentWaypointIndex`
Punkt przerwania	zapisany indeks z ostatniego cyklu
Stan stacji	`StationClient.status`
Obraz z kamery i marker	`CameraAdapter` + nakładka z `MarkerObservation`
Trasa patrolu na mapie	`PatrolRoute` + pozycja z `KeyAircraftLocation`
Etapy cyklu	podświetlony bieżący `CycleState`
Polecenia operatora	`CycleOrchestrator.startCycle()`, `.returnToStation()`
STOP	`SafetySupervisor.stop()` — duży przycisk, zawsze widoczny, bez potwierdzenia

Ekrany dodatkowe: **Planowanie trasy** (edycja punktów, walidacja stref, eksport KMZ) i **Stacja** (stan, ręczne polecenia, test popychacza). Ekran **Diagnostyka** wzorujemy na `KeyValueFragment.kt`: pozwala podejrzeć dowolny klucz MSDK na żywo.

Aplikacja działa jako **usługa pierwszoplanowa** (`foregroundService`) z wyłączoną optymalizacją baterii, żeby system nie zatrzymał jej w trakcie cyklu.

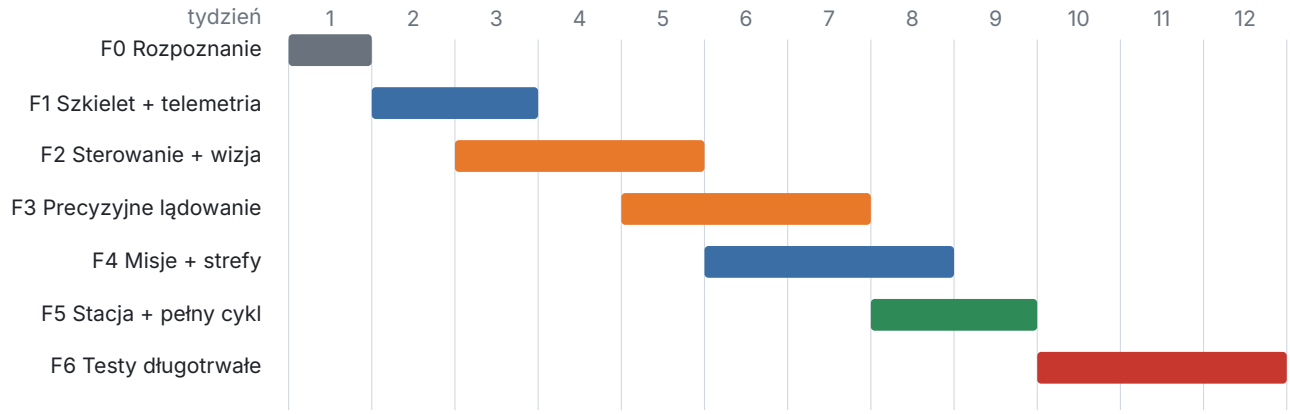
7 Testy wykonalności R1–R7

Testy z planszy D3. R1, R2 i R7 są ustalone lub sprawdzone; R3–R6 wymagają lotów. Każdy test w powietrzu najpierw przechodzi w symulatorze (`SimulatorManager.enableSimulator()`, wzorzec `SimulatorVM.kt`) z dronem bez śmigieł.

ID	Test	Metoda	Kryterium akceptacji
R1	Ładowanie akumulatora USTALONE	Ładowanie przy wyłączonym dronie. W aplikacji: `charge(on)` tylko w stanie WYŁĄCZANIE → ŁADOWANIE.	Pełne naładowanie bez interwencji; stacja raportuje koniec ładowania.
R2	Włączanie i wyłączanie USTALONE	Popychacz przycisku na stacji. Ustalić sekwencję naciśnięć i czas do `onProductConnect`.	Włączenie i połączenie z aplikacją w 20 próbach na 20; zmierzony czas uruchomienia.
R3	Sterowanie lotem z aplikacji TEST NA DRONIE	Virtual Stick w trybie zaawansowanym, pętla 20 Hz, watchdog. Najpierw symulator, potem otwarta przestrzeń na małej wysokości.	Zadana prędkość realizowana w obu osiach; STOP → zawis; watchdog działa po przerwaniu pętli.
R4	Opóźnienie obrazu TEST NA DRONIE	Kamera drona filmuje stoper na ekranie, nagrywamy jednocześnie stoper i podgląd w aplikacji. Mierzymy też czas detekcji ArUco.	Zmierzone opóźnienie całkowite (obraz + detekcja). Na tej podstawie ustalamy prędkości i wzmocnienia regulatora.
R5	Ochrona lądowania TEST NA DRONIE	Lądowanie na płycie stacji wywołane z aplikacji. Obsługa `KeyIsLandingConfirmationNeeded` → `KeyConfirmLanding`. Seria lądowań z markerem.	Dron ląduje na płycie bez odmowy; trafienie w pole styków w ustalonej tolerancji w co najmniej 19 z 20 prób.
R6	Misje po punktach TEST NA DRONIE	Trasa z aplikacji (KMZ) → `pushKMZFileToAircraft` → `startMission`. Pauza, wznowienie, przerwanie i wznowienie od punktu.	Trasa 6 punktów przelatana zgodnie z planem (log pozycji); wznowienie od punktu przerwania działa (natywnie lub rozwiązaniem zastępczym).
R7	Strefy lotów SPRAWDZONE	`FlyZoneManager` (wzorzec `FlySafeVM.kt`): sprawdzenie trasy przed wysłaniem; mapa stref i zgoda na lot.	Aplikacja blokuje misję w strefie zakazanej; procedura uzyskania zgody opisana.

8 Plan działania

Plan zakłada mały zespół (2–3 osoby). Czasy są orientacyjne; fazy częściowo się nakładają. Każda faza kończy się konkretnym rezultatem do pokazania.



F0. Rozpoznanie (tydzień 1)

- Konto deweloperskie DJI, klucz API, zbudowanie aplikacji przykładowej z repo.
- Połączenie z Mini 4 Pro przez RC-N2; spisanie wersji firmware drona i aparatury.
- Na ekranie Key-Value sprawdzenie, które klucze z tego dokumentu działają na Mini 4 Pro (szczególnie `KeyUltrasonicHeight`, `KeyConfirmLanding`, `KeyFailsafeAction`).
- Przejście ekranów Virtual Stick, WayPoint V3 i Simulator z przykładowej aplikacji.

Rezultat: raport zgodności „co działa na Mini 4 Pro". Decyzja o ewentualnych zmianach architektury.

F1. Szkielet aplikacji i telemetria (tygodnie 2–3)

- Projekt z modułami z sekcji 3.3, konfiguracja Gradle przeniesiona z repo.
- `SdkInitializer`, `DroneConnection`, `KeyRepository` z `asFlow()`, `DroneState`.
- Ekran główny: górny pasek, podgląd wideo, mapa z pozycją, przycisk STOP.
- Logowanie telemetrii do pliku od pierwszego dnia.

Rezultat: aplikacja łączy się z dronem, pokazuje obraz, pozycję i baterię.

F2. Sterowanie i wizja (tygodnie 3–5)

- `FlightControl` z pętlą 20 Hz i watchdogiem; testy w symulatorze, potem test R3.
- `ArucoDetector` rozwijany na nagraniach z drona; kalibracja kamery.
- Pomiar opóźnienia R4.

Rezultat: R3 i R4 zaliczone; dron w zawisie śledzi marker na ziemi bez lądowania.

F3. Precyzyjne lądowanie (tygodnie 5–7)

- Regulator PD i etapy L1–L5; strojenie najpierw na dużej wysokości nad markerem.
- Obsługa ochrony lądowania; test R5 na płycie stacji (najpierw bez stacji, tylko płyta).

Rezultat: R5 zaliczony: powtarzalne lądowanie w tolerancji styków.

F4. Misje i strefy (tygodnie 6–8)

- Ekran planowania trasy, `KmzBuilder`, `MissionAdapter`, punkt przerwania.
- Kontrola stref lotów (R7) przed wysłaniem misji; test R6.

Rezultat: R6 zaliczony: patrol z aplikacji z przerwaniem i wznowieniem.

F5. Stacja i pełny cykl (tygodnie 8–9)

- `StationClient` i firmware stacji (protokół z sekcji 5.7).
- `CycleOrchestrator`: wszystkie przejścia maszyny stanów, `SafetySupervisor`.

Rezultat: jeden pełny cykl GOTOWY → … → ŁADOWANIE → GOTOWY bez dotykania drona.

F6. Testy długotrwałe (tygodnie 10–12)

- Seria kolejnych cykli dzień po dniu; analiza logów, strojenie progów baterii.
- Testy awarii: zgaszenie stacji, wyłączenie Wi-Fi, zasłonięcie markera, STOP w każdym stanie.
- Procedura operacyjna i lista kontrolna przed lotem.

Rezultat: raport niezawodności i decyzja o dalszym rozwoju.

9 Ryzyka

Ryzyko	Ograniczenie
Funkcje MSDK opisane dla dronów Enterprise nie działają na Mini 4 Pro (punkt przerwania, część kluczy)	Faza F0 sprawdza zgodność; dla punktu przerwania mamy rozwiązanie zastępcze.
Brak omijania przeszkód w Virtual Stick	VS tylko przy lądowaniu nad wolnym lądowiskiem; patrol misją natywną.
Opóźnienie obrazu destabilizuje regulator	Pomiar R4, małe prędkości w końcowej fazie, opadanie tylko po wycentrowaniu.
Aktualizacja firmware drona lub aparatury psuje zgodność z MSDK	Blokada aktualizacji na sprzęcie produkcyjnym; porównanie z listą przetestowanych wersji w release notes MSDK.
Aparatura RC-N2 lub telefon wyłączy się albo rozładuje między cyklami	Stałe zasilanie aparatury i telefonu ze stacji; sprawdzić automatyczne wyłączanie RC-N2 przy braku drona.
System Android zatrzyma aplikację	Usługa pierwszoplanowa, wyłączona optymalizacja baterii, dedykowany telefon.
Ochrona lądowania odmawia lądowania na płycie	Płaska, kontrastowa płyta; `KeyConfirmLanding`; test R5.
Wiatr (Mini 4 Pro waży poniżej 250 g)	Limit wiatru w procedurze operacyjnej; większa tolerancja i więcej prób przy lądowaniu.
Regulacje lotnicze	Automatyczny lot bez stałej obserwacji wzrokowej zwykle wykracza poza kategorię otwartą. Przed lotami produkcyjnymi trzeba sprawdzić wymagania ULC (kategoria szczególna) i zasady zgłaszania lotów w strefach.

A Ściąga API MSDK użyte w projekcie

API	Zastosowanie
`SDKManager.init/registerApp`	start MSDK
`KeyManager.listen/getValue/performAction`	telemetria i akcje
`KeyTools.createKey(...)`	tworzenie kluczy
`VirtualStickManager.enableVirtualStick`	włączenie sterowania z aplikacji
`.setVirtualStickAdvancedModeEnabled`	tryb zaawansowany
`.sendVirtualStickAdvancedParam`	polecenia ruchu (5–25 Hz)
`.setVirtualStickStateListener`	zmiana właściciela sterowania
`WPMZManager.generateKMZFile`	budowa misji
`WaypointMissionManager.pushKMZFileToAircraft`	wysłanie misji
`.startMission/pauseMission/resumeMission/stopMission`	sterowanie misją
`.addWaylineExecutingInfoListener`	bieżący punkt trasy
`CameraStreamManager.addFrameListener`	klatki wideo (NV21)
`FlightControllerKey.KeyStartTakeoff/KeyStartAutoLanding`	start i lądowanie
`KeyIsLandingConfirmationNeeded/KeyConfirmLanding`	ochrona lądowania
`GimbalKey.KeyRotateByAngle`	kamera w dół
`SimulatorManager.enableSimulator`	testy bez lotu
`FlyZoneManager`	strefy lotów

Nazwy klas i kluczy pochodzą z kodu i dokumentacji w repozytorium `dji-sdk/Mobile-SDK-Android-V5` (MSDK 5.18.0). Fragmenty kodu w dokumencie to szkice architektoniczne, nie gotowy kod produkcyjny.